

Handling Service Failures in Microservices

Learning Objectives

After completing this chapter, you will be able to:

- Understand why service failures happen.
- Learn different types of failures in distributed systems.
- Configure connection and read timeouts.
- Implement Retry mechanisms.
- Understand and implement the Circuit Breaker pattern.
- Create Fallback methods.
- Learn Bulkhead and Rate Limiter concepts.
- Use **Resilience4j** with Spring Boot.

Why Do Service Failures Occur?

In a monolithic application, all modules run inside the same process. Communication happens through direct method calls.

```
CustomerService.getCustomer()
```

↓

```
OrderService.createOrder()
```

Method calls are fast and reliable.

In a microservices architecture, communication happens over the network.

Order Service



HTTP Request



Internet / Network



Customer Service

Unlike local method calls, network communication introduces uncertainty.

Several things can go wrong:

- Network latency

Example Scenario

Suppose a customer places an order.

User



POST /orders



Order Service



GET /customers/101



Customer Service

Common Types of Service Failures

1. Connection Failure

Order Service cannot establish a connection.

Example:

Connection Refused

Possible reasons:

- Service is not running.
- Wrong host or port.
- Firewall blocks communication.

2. Read Timeout

Connection is established, but the response takes too long.

Example:

Order Service

↓

Waiting...

↓

Waiting...

↓

Timeout

3. HTTP Errors

Customer Service responds with an error.

Examples:

404 Not Found

500 Internal Server Error

503 Service Unavailable

4. Slow Service

Customer Service is working but is overloaded.

Normally:

200 ms

Today:

20 seconds

Without timeouts, Order Service keeps waiting.

Why Timeouts Are Important

Imagine calling a friend.

If they don't answer, would you wait forever?

Of course not.

You wait a few seconds and hang up.

The same principle applies to microservices.

Types of Timeouts

Connection Timeout

Maximum time allowed to establish the TCP connection.

Order Service



Connect



3 Seconds



Failed

Read Timeout

Maximum time allowed to wait for the response after the connection is established.

Connected



Waiting for Data



5 Seconds



Timeout

Configuring Timeouts (RestClient Example)

```
@Bean
RestClient restClient() {

    HttpClient httpClient = HttpClient.newBuilder()
        .connectTimeout(Duration.ofSeconds(3))
        .build();

    return RestClient.builder()
        .requestFactory(
            new JdkClientHttpRequestFactory(httpClient)
        )
        .build();
}
```

Here:

- Connection timeout = 3 seconds
- If the connection isn't established within 3 seconds, the request fails

Retry Pattern

Sometimes failures are temporary.

Example:

Attempt 1



Network Error



Attempt 2



Success

Real-Life Example

Suppose Customer Service is restarting.

Attempt 1

↓

Failed

↓

2 Seconds Later

↓

Attempt 2

↓

Customer Service Started

↓

When Should You Retry?

Good situations:

- Temporary network issues.
- Service startup.
- Short server overload.
- Temporary database locks.

Avoid Retry for:

- Validation errors.
- Authentication failures.
- Invalid customer IDs.
- Business rule violations.

Resilience4j

Resilience4j is a lightweight Java library that adds fault tolerance to your application.

It provides:

- Retry
- Circuit Breaker
- Rate Limiter
- Bulkhead
- Time Limiter
- Cache

It is the recommended replacement for the older Hystrix library.

Maven Dependency

```
<dependency>  
  <groupId>io.github.resilience4j</groupId>  
  <artifactId>resilience4j-spring-boot3</artifactId>  
</dependency>
```

Retry Example

```
@Retry(name = "customerService")  
public Customer getCustomer(Long id) {  
    return customerClient.getCustomer(id);  
}
```

If the request fails,

Spring automatically retries based on configuration.

Retry Configuration

```
resilience4j.retry.instances.customerService.max-attempts=3
```

```
resilience4j.retry.instances.customerService.wait-duration=2s
```

Meaning:

Attempt 1

↓

Failed

↓

Wait 2 Seconds

↓

Attempt 2

↓

Failed

↓

What is a Circuit Breaker?

Imagine a circuit breaker in your home.

When electrical overload occurs,
the breaker opens to prevent damage.

The same concept applies to software.

Without Circuit Breaker

Order Service



Customer Service Down



Retry



Retry



Retry



Retry



Retry



System Overload

With Circuit Breaker

Customer Service Down



Circuit Opens



Requests Blocked



Immediate Response



Try Again Later

This protects the application.

Circuit Breaker States

Closed

Everything is healthy.

Requests are allowed.

Requests



Customer Service



Success

Open

Too many failures detected.

Requests are blocked.

Requests



Circuit Breaker



Rejected Immediately

Half-Open

After waiting for some time,
the Circuit Breaker allows a few requests.

One Request



Customer Service



Success?



Close Circuit

or

Circuit Breaker Example

```
@CircuitBreaker(  
    name = "customerService",  
    fallbackMethod = "customerFallback"  
)  
public Customer getCustomer(Long id) {  
    return customerClient.getCustomer(id);  
}
```

Fallback Method

When all retries fail,

Spring calls another method.

```
public Customer customerFallback(  
    Long id,  
    Exception ex){  
  
    Customer customer = new Customer();  
  
    customer.setId(id);  
  
    customer.setName("Unknown Customer");  
  
    customer.setEmail("Service Unavailable");  
  
    return customer;  
  
}
```

Flow with Circuit Breaker

Order Service



Customer Service



Failure



Retry



Failure



Circuit Opens



Fallback Method



Return Default Response

Bulkhead Pattern

Suppose a ship has multiple watertight compartments.

If one compartment floods,
the entire ship does not sink.

Software follows the same principle.

Different operations use separate thread pools or resources.

Example:

Payment Service



Dedicated Thread Pool

Rate Limiter

Sometimes a service receives too many requests.

Example:

10000 Requests

↓

Customer Service

↓

Crash

Rate Limiter controls traffic.

Example:

Complete Failure Flow

User



Order Service



Customer Client



Retry



Still Failed



Circuit Breaker



Fallback



Default Response



Application Continues Running

Best Practices

- ✓ Always configure connection and read timeouts.
- ✓ Retry only temporary failures.
- ✓ Never retry validation or authentication errors.
- ✓ Use Circuit Breaker to prevent cascading failures.
- ✓ Return meaningful fallback responses.
- ✓ Log failures with enough detail for debugging.
- ✓ Monitor retry and circuit breaker metrics using Spring Boot Actuator.
- ✓ Test failure scenarios regularly, not just successful paths.

Common Mistakes

- ✗ Waiting forever for a response.
- ✗ Retrying every type of error.
- ✗ Returning `null` in fallback methods.
- ✗ Catching generic `Exception` without proper logging.
- ✗ Ignoring timeout configuration.
- ✗ Not monitoring resilience metrics.

Interview Questions

1. Why do service failures happen in microservices?

Because communication occurs over the network, which is slower and less reliable than local method calls. Failures may be caused by network issues, server crashes, slow responses, or infrastructure problems.

2. What is the difference between Connection Timeout and Read Timeout?

- **Connection Timeout:** Maximum time to establish the connection.
- **Read Timeout:** Maximum time to wait for data after the connection is established.

3. What is Retry?

Retry automatically attempts a failed operation again after a configured delay, helping recover from temporary failures.

4. What is a Circuit Breaker?

A Circuit Breaker stops sending requests to a failing service after repeated failures, preventing resource exhaustion and allowing the service time to recover.

5. What are the three states of a Circuit Breaker?

- Closed
- Open
- Half-Open

6. What is a Fallback Method?

A fallback method provides an alternative response when the main service call fails, allowing the application to continue functioning gracefully.

7. What is the Bulkhead Pattern?

The Bulkhead pattern isolates resources (such as thread pools) so that failures in one part of the application do not affect other parts.

8. What is Rate Limiting?

Rate Limiting restricts the number of requests a client can make within a specific time period, protecting services from overload.

Summary

In this chapter, you learned:

- Why service failures occur in distributed systems.
- Different categories of service failures.
- The importance of connection and read timeouts.
- How Retry handles temporary failures.
- How Circuit Breaker prevents cascading failures.
- How Fallback methods improve user experience.
- The purpose of Bulkhead and Rate Limiter patterns.
- How **Resilience4j** helps build resilient, production-ready microservices.
- Enterprise best practices and common interview questions.

Next Chapter: Spring Boot Actuator (Production Monitoring)

This will be the final chapter for this topic and will cover:

- What is Spring Boot Actuator?
- Production-ready endpoints.
- `/actuator/health`, `/metrics`, `/info`, `/beans`, `/env`, `/mappings`.
- Custom Health Indicators.
- Health Groups.
- Micrometer integration.
- Prometheus and Grafana basics.
- Monitoring in Kubernetes.
- Security of Actuator endpoints.
- Complete Spring Boot project with code.